

Homework

One

1

Так как у нас тип данных Unsigned и нужно перевести из десятичной в двоичную (важно: В двоичной по условию 6 битов $\Rightarrow 2^6 = 64$), то наш тип содержит числа в промежутке $[0, 63]$. Теперь решения:

$$\begin{aligned}0 &= 0 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 0 * 2^1 + 0 * 2^0 \rightarrow 000000 \\13 &= 0 * 2^5 + 0 * 2^4 + 1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 \rightarrow 001101 \\24 &= 0 * 2^5 + 1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 0 * 2^1 + 0 * 2^0 \rightarrow 011000 \\63 &= 1 * 2^5 + 1 * 2^4 + 1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0 \rightarrow 111111\end{aligned}$$

Signed содержит как положительные, так и отрицательные числа; в нашем случае это промежуток $[-2^5, 2^5 - 1] \rightarrow [-32, 31]$

Суть алгоритма проста: с положительными работаем как с unsigned, с отрицательными – так: у них на первом бите всегда 1 (у положительных 0). запоминаем эту информацию, потом берем данное отрицательное число по модулю и работаем с ним как с unsigned, а после преобразования инвертируем все биты и прибавляем 1.

$$\begin{aligned}16 &= 0 * 2^5 + 1 * 2^4 + 0 * 2^3 + 0 * 2^2 + 0 * 2^1 + 0 * 2^0 \rightarrow 010000 \\-2 &\rightarrow |2| = 0 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 \rightarrow 000010 \\&\rightarrow 111101 \rightarrow 111110 \\31 &= 0 * 2^5 + 1 * 2^4 + 1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0 \rightarrow 011111 \\-32 &\rightarrow |32| = 1 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 0 * 2^1 + 0 * 2^0 \rightarrow 100000 \\&\rightarrow 011111 \rightarrow 100000\end{aligned}$$

2

Теперь нужно перевести из двоичной в десятичную для Unsigned и Signed сразу, то есть проходим путь назад (для значений, начинающихся с 0 будем писать один тип, т.к. они одинаковы (оба только положительные)):

$$\begin{aligned}000101 &\rightarrow 0 * 2^5 + 0 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 \rightarrow 5 \\Unsigned: 101011 &\rightarrow 1 * 2^5 + 0 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 \rightarrow 43 \\Signed: 101011 &\rightarrow 101010 \rightarrow 010101 \rightarrow 0 * 2^5 + 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 \rightarrow |21| \rightarrow -21\end{aligned}$$

1

Unsigned: $111111 \rightarrow 1 * 2^5 + 1 * 2^4 + 1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0 \rightarrow 63$

Signed: $111111 \rightarrow 111110 \rightarrow 000001 \rightarrow 0 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 0 * 2^1 + 1 * 2^0 \rightarrow |1| \rightarrow -1$

Unsigned: $100000 \rightarrow 1 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 0 * 2^1 + 0 * 2^0 \rightarrow 32$

Signed: $100000 \rightarrow 011111 \rightarrow 100000 \rightarrow 1 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 0 * 2^1 + 0 * 2^0 \rightarrow |32| \rightarrow -32$

3

Нужно перевести число сначала в бинарное 8-битное, потом разделить получившееся на 2 блока по 4 бита и перевести каждый блок в шестнадцатичную:

$7 \rightarrow 1110 \rightarrow 00001110 \rightarrow 0x07$

$240 \rightarrow 11110000 \rightarrow 0xF0$

$171 \rightarrow 10101011 \rightarrow 0xAB$

$126 \rightarrow 1111110 \rightarrow 01111110 \rightarrow 0x7E$

4

Теперь надо шестнадцатичную преобразовать в 8-битную:

$0x3C \rightarrow 00111100$

$0x7E \rightarrow 01111110$

$0xFF \rightarrow 11111111$

$0xA5 \rightarrow 10100101$

5

Просто инвертирование битов из прошлого задания а потом прибавление единицы к получившемуся результату

$00111100 \rightarrow 11000011 \rightarrow 11000100$

$01111110 \rightarrow 10000001 \rightarrow 10000010$

$11111111 \rightarrow 00000000 \rightarrow 00000001$

$10100101 \rightarrow 01011010 \rightarrow 01011011$

6

Big-endian — старший байт по меньшему адресу:

$0xDEADBEEF \rightarrow DE(0) AD(1) BE(2) EF(3) \rightarrow DE(0) AD(1) BE(2) EF(3)$ (то есть ничего не изменилось)

Little-endian — младший байт по меньшему адресу:

$0xDEADBEEF \rightarrow DE(0) AD(1) BE(2) EF(3) \rightarrow EF(3) BE(2) AD(1)$

DE(0)

7

Сначала инвертируем в 5-битное значение, а потом либо в начало добавляем нулей до 8-битного значения(zero-), либо повторяем старший бит до 8-битного значения(тоже добавляем в начало)(sign-)

$7 \rightarrow 00111 \rightarrow 00000111(\text{zero-}) \rightarrow 00000111(\text{sign-})$

$15 \rightarrow 01111 \rightarrow 00001111(\text{zero-}) \rightarrow 00001111(\text{sign-})$

$-16 \rightarrow 10000 \rightarrow 00010000(\text{zero-}) \rightarrow 11110000(\text{sign-})$

$-5 \rightarrow 11011 \rightarrow 00011011(\text{zero-}) \rightarrow 11111011(\text{sign-})$

8

$7 + 9 \rightarrow 0111 + 1001 \rightarrow 10000(5\text{-бит}) \rightarrow 0000 = 0$ (проверка: $7 + 9 = 16$ (но т.к. у нас $[0,15]$, то можно считать $\text{mod } 16 \rightarrow 16 \text{ mod } 16 = 0$)

$4 + (-5) \rightarrow 0100 + 1011 \rightarrow 1111 \rightarrow -1$ (проверка: $4 + (-5) = -1$)

3